build.gradle.kts 解説 — Kotlin と Gradle の基礎から TeaVM の設定まで

このレポートは、`make_wasm/TeaVM/app/build.gradle.kts` を **1 行ずつ読めるようになること**を目的にしています。あわせて、その前提となる **Gradle（ビルドツール）** と **Kotlin（プログラミング言語）** の基本、Kotlin で何ができるかまでを 1 本にまとめました。

想定する読み手は、Java はある程度書けるが、Gradle や Kotlin はほとんど触ったことがない人です。

- 重要度の凡例：★★★ まず必要 / ★★ 知っておくと困らない / ★ 余裕があれば
- 確認環境：Gradle 8.14.4（同梱 Kotlin 2.0.21）、JDK 21、TeaVM Gradle プラグイン 0.16.0（2026 年 11 月時点で実機確認）
- 載せた Kotlin のコード例と Gradle タスクの例は、実際に Gradle 上で実行して出力を確かめています。

目次

1. 全体像：build.gradle.kts は何をしているファイルか
2. Gradle とは何か・どんな場面で使うか
3. このプロジェクトの Gradle 関連ファイル
4. Gradle の動き方：プロジェクト・タスク・ライフサイクル
5. Kotlin の基本文法
6. Kotlin DSL を読むための Kotlin の機能
7. build.gradle.kts を 1 ブロックずつ読む
8. plugins ブロック：java・war・org.teavm
9. repositories と dependencies：ライブラリの取り込み
10. java toolchain と JDK の指定
11. teavm ブロック：Wasm を作る設定
12. カスタムタスク parseJava
13. ./gradlew :app:build で実際に起きること
14. Groovy DSL との対応表
15. よく使う Gradle コマンド集
16. このビルドを拡張する例
17. Kotlin で何ができるか
18. よくある失敗・ハマりどころ
19. 学習の進め方と参考資料
20. チェックリスト
21. まとめ

---

1. 全体像：build.gradle.kts は何をしているファイルか

★★★

`build.gradle.kts` は、**Gradle というビルドツールに「このプロジェクトをどう組み立てるか」を教える設定ファイル**です。 拡張子の `.kts` は「Kotlin Script（Kotlin で書いたスクリプト）」の意味で、中身は **Kotlin のプログラム**です。

まず、このプロジェクトのファイル全体を見ておきます（58 行）。

```
plugins {
    java
    war
    id("org.teavm") version "0.16.0"
}

repositories {
    mavenCentral()
}

dependencies {
    implementation(teavm.libs.jsoApis)
    // JavaParser/JavaStructureAnalyzer で使用（TeaVM の Wasm には含まれない）
    implementation("com.github.javaparser:javaparser-core:3.26.4")
    // JavaParser/JavaApiJsonExporter で JSON を書き出すために使用
    implementation("com.google.code.gson:gson:2.11.0")
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

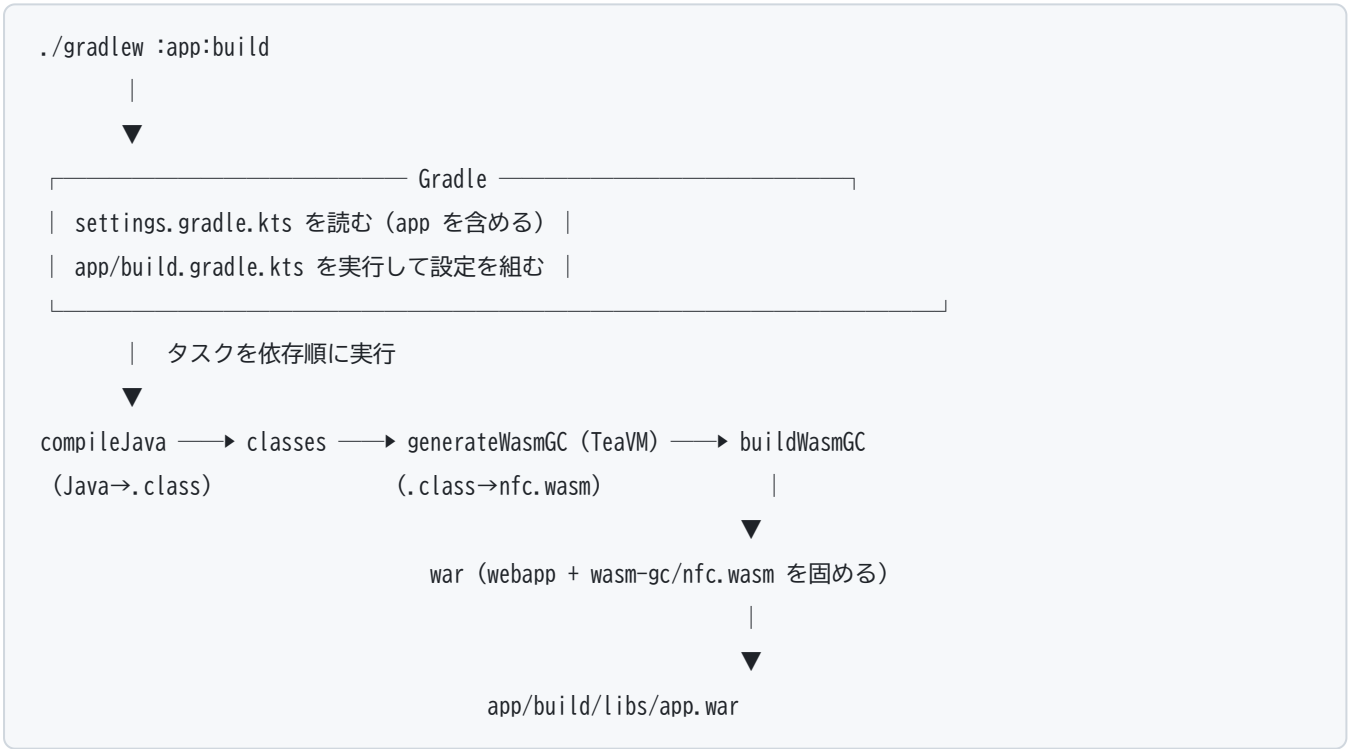
teavm {
    all {
        // JS へ公開する入口。org.mklab.nfc の必要なクラスは TeaVM が依存関係から自動で取り込む
        mainClass = "org.example.NfcWasm"
    }
    wasmGC {
        addedToWebApp = true
        targetFileName = "nfc.wasm"
    }
}

tasks.register<JavaExec>("parseJava") {
    group = "application"
    description = "JavaParser で指定フォルダ内の .java ファイルの構造を表示する"
    classpath = sourceSets["main"].runtimeClasspath
    mainClass = "JavaParser.JavaStructureAnalyzer"
    workingDir = rootProject.projectDir
}
```

各ブロックの役割をひとことで

ブロック	ひとことで	詳しくは
`plugins { }`	使う機能（プラグイン）を宣言する。Java のコンパイル、war 作成、TeaVM 変換	8 章
`repositories { }`	ライブラリをどこからダウンロードするか（Maven Central）	9 章
`dependencies { }`	使うライブラリの一覧（TeaVM の JS 連携 API、JavaParser、Gson）	9 章
`java { toolchain { } }`	Java 21 でコンパイルする	10 章
`teavm { }`	Java → WebAssembly 変換の設定。入口クラスと出力ファイル名	11 章
`tasks.register<JavaExec>("parseJava")`	自作のコマンド `parseJava` を追加する	12 章

全体の流れ（図）



つまりこのファイルは、「**Java のソースをコンパイル → TeaVM で `nfc.wasm` に変換 → `index.html`・`nfc.js` と一緒に war ファイルへまとめる**」という流れを、わずか 58 行で宣言しています。 手順（どう作るか）を全部書くのではなく、**何がほしいか**を書けばプラグインが手順を用意してくれる、というのが Gradle の考え方です。

---

2. Gradle とは何か・どんな場面で使うか

★★★

ビルドツールとは

ビルド（build）とは、ソースコードから「実行できる成果物」を作る一連の作業のことです。Java なら次のような作業が含まれます。

1. 必要なライブラリ（jar）をダウンロードする
2. `.java` を `.class` にコンパイルする
3. テストを実行する
4. jar / war などにまとめる
5. （このプロジェクトでは）TeaVM で `.wasm` に変換する

これを毎回手で `javac -cp ...` と打つのは大変で、間違いも起きます。**ビルドツール**はこれを自動化するソフトです。Gradle はその代表の 1 つです。

Gradle の特徴

特徴	説明
依存関係の管理	`dependencies` に書くだけで、ライブラリとその依存ライブラリ（推移的依存）を自動でダウンロードする
タスクの自動実行	「war を作る」と言えば、その前に必要なコンパイルなどを依存順に自動で実行する
差分ビルド（インクリメンタルビルド）	入力が変わっていないタスクは `UP-TO-DATE` としてスキップし、速い
ビルドキャッシュ・構成キャッシュ	前回の結果を再利用してさらに速くする（このプロジェクトでは構成キャッシュが有効）
プラグイン	Java・Android・Kotlin・TeaVM など、機能をプラグインで追加できる
ビルドスクリプトがプログラム	Kotlin（または Groovy）で書くので、条件分岐や自作タスクも自由に書ける
Gradle Wrapper	`gradlew` を使えば、Gradle をインストールしていない PC でも決まったバージョンで動く

Gradle を使う場面

場面	例
Java / Kotlin のアプリやライブラリ開発	このプロジェクト、Spring Boot のサーバー、CLI ツール
Android アプリ開発	Android Studio のプロジェクトは標準で Gradle（`build.gradle.kts`）
Kotlin Multiplatform	1 つのコードから JVM・Android・iOS・JS・Wasm 向けにビルド
Java → 別ターゲットへの変換	TeaVM（Java → JS / Wasm）、GraalVM Native Image など
複数モジュールの大きなプロジェクト	`settings.gradle.kts` で複数のサブプロジェクトをまとめる
CI（自動テスト・自動ビルド）	GitHub Actions などで `./gradlew build` を実行する
定型作業の自動化	コード生成、ファイルのコピー、外部コマンド実行などを自作タスクにする

他のビルドツールとの比較

ツール	設定の書き方	特徴
Gradle	Kotlin / Groovy のスクリプト	柔軟・高速。Android の標準
Maven	XML（`pom.xml`）	決まった流れに沿うので読みやすいが、柔軟性は低め。Java の古くからの定番
Ant	XML（`build.xml`）	手順をすべて自分で書く。古いプロジェクトで見かける
Make	`Makefile`	C/C++ の定番。依存ライブラリの管理機能はない
npm / Vite	`package.json` など	JavaScript の世界のビルド・依存管理

Gradle は Maven の「リポジトリ」と「座標（`group:artifact:version`）」の仕組みをそのまま使っています。そのため、Maven 向けに公開されたライブラリは Gradle からも使えます。

---

3. このプロジェクトの Gradle 関連ファイル

★★★

`make_wasm/TeaVM/` の Gradle 関連ファイルは次のとおりです。

```
make_wasm/TeaVM/
├── settings.gradle.kts          ← どのサブプロジェクトを含めるか
├── gradle.properties            ← Gradle 自体の設定（JDK の場所など）
├── gradlew / gradlew.bat        ← Gradle Wrapper の起動スクリプト
├── gradle/
│   ├── wrapper/
│   │   ├── gradle-wrapper.jar
│   │   └── gradle-wrapper.properties   ← 使う Gradle のバージョン（8.14.4）
│   └── libs.versions.toml       ← バージョンカタログ（現状は未使用）
└── app/
    ├── build.gradle.kts         ← ★このレポートの主役
    └── src/main/
        ├── java/                ← Java ソース
        └── webapp/              ← war に入る静的ファイル（index.html など）
```

settings.gradle.kts

```
plugins {
    // JDK を自動ダウンロードできるようにするプラグイン
    id("org.gradle.toolchains.foojay-resolver-convention") version "0.10.0"
}

rootProject.name = "kogalab"
include("app")
```

- `rootProject.name`：ルートプロジェクトの名前。
- `include("app")`：`app/` フォルダをサブプロジェクトとして含める。だからコマンドは `./gradlew :app:build` のように `:app:` を付ける。
- `foojay-resolver-convention`：toolchain（10 章）で指定した JDK が手元にないとき、自動でダウンロードする仕組み。

gradle.properties

```
org.gradle.configuration-cache=true
org.gradle.java.home=/usr/lib/jvm/java-21-openjdk-amd64
```

- `org.gradle.configuration-cache=true`：**構成キャッシュ**を有効にする。2 回目以降、ビルドスクリプトの実行（設定フェーズ）を省略して速くなる。
- `org.gradle.java.home`：**Gradle 自身**を動かす JDK の場所。WSL の Ubuntu にある JDK 21 を指している。

gradle/wrapper/gradle-wrapper.properties

```
distributionUrl=https¥://services.gradle.org/distributions/gradle-8.14.4-bin.zip
```

`./gradlew` を実行すると、このファイルに書かれた Gradle 8.14.4 が（なければ自動でダウンロードされて）使われます。チーム全員が同じバージョンでビルドできるのが利点です。

gradle/libs.versions.toml（バージョンカタログ）

```
[versions]
guava = "33.4.5-jre"
junit-jupiter = "5.12.1"

[libraries]
guava = { module = "com.google.guava:guava", version.ref = "guava" }
junit-jupiter = { module = "org.junit.jupiter:junit-jupiter", version.ref = "junit-jupiter" }
```

`gradle init` が作った雛形で、**このプロジェクトの build.gradle.kts からは参照されていません**（guava も junit も使っていない）。 使う場合は `implementation(libs.guava)` のように書けます。ライブラリのバージョンを 1 か所で管理したいときに便利です（16 章で例を示します）。

各ファイルの比較

ファイル	書く言語	役割	このプロジェクトで編集するか
`settings.gradle.kts`	Kotlin	プロジェクト構成	ほぼしない
`app/build.gradle.kts`	Kotlin	ビルドの中身	**する**
`gradle.properties`	properties	Gradle の動作設定	JDK を変えるとき
`gradle-wrapper.properties`	properties	Gradle のバージョン	更新するときだけ（`./gradlew wrapper` で）
`libs.versions.toml`	TOML	バージョン一覧	使うなら

---

4. Gradle の動き方：プロジェクト・タスク・ライフサイクル

★★★

用語

用語	意味	このプロジェクトでは
プロジェクト（project）	ビルドの単位。1 つの `build.gradle.kts` が 1 プロジェクト	ルート `kogalab` とサブ `:app`
タスク（task）	実行できる作業の単位	`compileJava`、`war`、`generateWasmGC`、`parseJava` など
プラグイン（plugin）	タスクや設定をまとめて追加する部品	`java`、`war`、`org.teavm`
拡張（extension）	プラグインが追加する設定ブロック	`java { }`、`teavm { }`
構成（configuration）	依存ライブラリのグループ	`implementation`、`runtimeClasspath`
ソースセット（source set）	ソースのまとまり	`main`、`test`、TeaVM が追加する `teavm`

ビルドの 3 つのフェーズ

Gradle は、どのビルドでも次の 3 段階で動きます。

フェーズ	すること	このプロジェクトでは
1. 初期化（Initialization）	`settings.gradle.kts` を読み、どのプロジェクトがあるか決める	`kogalab` と `app`
2. 設定（Configuration）	各 `build.gradle.kts` を**上から実行**し、タスクとその設定を組み立てる	`plugins` 〜 `tasks.register` が実行される
3. 実行（Execution）	指定されたタスクと、それが依存するタスクを順番に実行する	`compileJava` → … → `war`

ここで大事なのは、**build.gradle.kts の中身は「設定フェーズ」で実行される**という点です。
`tasks.register(...) { ... }` の中に `println` を書くと設定時に表示されますが、実際の作業は `doLast { }` の中に書かないと「実行フェーズ」で動きません（16 章の例を参照）。

タスクグラフ

タスク同士には「A の前に B が必要」という依存関係があり、Gradle はこれを**有向非巡回グラフ（DAG）**として組み立てます。 `./gradlew war` と打つと、`war` が必要とする `compileJava` や `buildWasmGC` も自動で先に実行されます。どの順で動くかは `--dry-run` で確かめられます（13 章）。

UP-TO-DATE（差分ビルド）

各タスクは「入力（ソースなど）」と「出力（.class など）」を Gradle に申告しています。 前回から入力も出力も変わっていなければ、タスクは `UP-TO-DATE` と表示されてスキップされます。2 回目の `./gradlew :app:build` が速いのはこのためです。

---

5. Kotlin の基本文法

★★★

Kotlin は JetBrains 社が開発した言語で、**JVM（Java の実行環境）上で動き、Java と自由に混ぜて使えます。** Java より短く安全に書けるのが特徴です。 ここでは build.gradle.kts を読むのに必要な範囲を中心に、基本文法をまとめます。以下のコードはすべて実際に実行して確かめたもので、コメントの `// →` は出力です。

5.1 変数：val と var

```
val name: String = "NFC"   // val = 再代入できない（Java の final）
var count = 0              // var = 再代入できる。型は Int と推論される
count += 1
println("$name $count ${name.length}")   // → NFC 1 3
```

Kotlin	Java	意味
`val x = 1`	`final int x = 1;`	読み取り専用
`var x = 1`	`int x = 1;`	変更できる
`val s: String`	`String s`	型は**名前の後ろ**に `: 型` で書く

- 行末の `;` は不要です。
- 型は多くの場合**推論**されるので書かなくてよいです。
- まず `val` を使い、必要なときだけ `var` にするのが Kotlin の習慣です。

5.2 文字列テンプレート

```
println("$name $count ${name.length}")
```

`"$変数"` や `"${式}"` で、文字列の中に値を埋め込めます。Java の `"" + name + " " + count` より読みやすくなります。

複数行の文字列は `"""` で囲みます。

```
val raw = """
    |行1
    |行2
""".trimMargin()
println(raw.lines())   // → [行1, 行2]
```

5.3 関数

```
fun add(a: Int, b: Int): Int {
    return a + b
}
fun mul(a: Int, b: Int) = a * b          // 1 行なら = で書ける（戻り値の型も推論）
fun greet(who: String = "TeaVM", times: Int = 1) = "Hello, $who! ".repeat(times)

println(add(2, 3))                       // → 5
println(greet())                         // → Hello, TeaVM!
println(greet(times = 2, who = "Wasm"))  // → Hello, Wasm! Hello, Wasm!
```

- `fun 名前(引数: 型): 戻り値の型` の形。
- **デフォルト引数**（`= "TeaVM"`）と**名前付き引数**（`times = 2`）が使えるので、Java のようにオーバーロードを何個も作らなくて済みます。

5.4 null 安全

Kotlin は「null になりうる型」と「ならない型」を区別します。これが Java との一番大きな違いです。

```
val maybe: String? = null       // ? を付けると null を入れられる
println(maybe?.length)          // → null    (?. は null なら呼ばずに null を返す)
println(maybe?.length ?: 0)     // → 0       (?: は null のときの代わりの値。エルビス演算子)
```

書き方	意味
`String`	null を入れられない（入れようとするとコンパイルエラー）
`String?`	null を入れられる
`a?.b`	a が null なら null、そうでなければ `a.b`
`a ?: b`	a が null なら b
`a!!`	「絶対 null じゃない」と断言（null なら例外）。なるべく使わない

`NullPointerException` の多くがコンパイル時に防げるのが利点です。

5.5 if と when は「式」

```
val n = 7
val parity = if (n % 2 == 0) "偶数" else "奇数"   // if が値を返す（三項演算子の代わり）
val kind = when (n) {                             // when は Java の switch の強化版
    0 -> "ゼロ"
    in 1..9 -> "1 桁"                             // 範囲での条件も書ける
    else -> "それ以外"
}
println("$parity $kind")   // → 奇数 1 桁
```

5.6 コレクションとラムダ

```
val nums = listOf(1, 2, 3, 4, 5)          // 読み取り専用リスト（mutableListOf なら変更可）
val evens = nums.filter { it % 2 == 0 }    // → [2, 4]
val squares = nums.map { x -> x * x }      // → [1, 4, 9, 16, 25]
val total = nums.sum()                     // → 15
val m = mapOf("rows" to 2, "cols" to 3)    // Map。"キー" to 値 でペアを作る
println(m["rows"])                         // → 2    ([ ] で取り出せる)

for (i in 0 until 3) print("$i ")          // → 0 1 2
```

- `{ x -> x * x }` が**ラムダ式**（名前のない関数）です。引数が 1 つなら `it` という名前で省略できます。
- `0 until 3` は 0, 1, 2、`1..3` は 1, 2, 3（端を含む）です。

5.7 クラスと data class

```
data class Matrix(val rows: Int, val cols: Int, val data: List<Double>)

val a = Matrix(2, 2, listOf(1.0, 2.0, 3.0, 4.0))   // new は不要
val b = a.copy(data = listOf(0.0, 0.0, 0.0, 0.0))  // 一部だけ変えたコピー
println(a)            // → Matrix(rows=2, cols=2, data=[1.0, 2.0, 3.0, 4.0])
println(a == Matrix(2, 2, listOf(1.0, 2.0, 3.0, 4.0)))   // → true（中身で比較）
```

`data class` と書くだけで、コンストラクタ・getter・`equals`・`hashCode`・`toString`・`copy` が自動で作られます。Java で書くと数十行になる部分が 1 行です。

5.8 Java との対応表（まとめ）

やりたいこと	Java	Kotlin
変数	`final String s = "a";`	`val s = "a"`
関数	`int f(int x) { return x*2; }`	`fun f(x: Int) = x * 2`
インスタンス生成	`new Foo()`	`Foo()`
文字列連結	`"n=" + n`	`"n=$n"`
三項演算子	`c ? a : b`	`if (c) a else b`
switch	`switch (x) { case 1: ... }`	`when (x) { 1 -> ... }`
null チェック	`s != null ? s.length() : 0`	`s?.length ?: 0`
static メソッド	`static void f()`	トップレベル関数 `fun f()` か `companion object`
型チェックとキャスト	`if (o instanceof String) ((String) o).length()`	`if (o is String) o.length`（スマートキャスト）
ゲッター呼び出し	`p.getName()`	`p.name`（Java の getter もプロパティとして呼べる）

最後の行は Gradle を読むうえで重要です。Gradle の API は Java で書かれていますが、Kotlin からは `getGroup()` / `setGroup()` を **`group`** というプロパティとして扱えます。

---

6. Kotlin DSL を読むための Kotlin の機能

★★★

DSL（ドメイン固有言語）とは、特定の用途に特化した「ミニ言語」のことです。Gradle の Kotlin DSL は、Kotlin の文法を使って「設定ファイルらしく」見えるように作られています。 build.gradle.kts が普通の Kotlin に見えないのは、次の機能を組み合わせているからです。

6.1 末尾ラムダ（trailing lambda）

関数の最後の引数がラムダなら、`( )` の外に出せます。引数がラムダだけなら `( )` ごと省略できます。

```
fun repeatTwice(action: () -> Unit) { action(); action() }

repeatTwice({ print("hi ") })   // 普通の書き方
repeatTwice { print("hi ") }    // 末尾ラムダ。→ hi hi
```

つまり `repositories { mavenCentral() }` は、**`repositories` という関数に `{ mavenCentral() }` というラムダを渡している**のです。

6.2 レシーバ付きラムダ（lambda with receiver）

これが DSL の核心です。ラムダの中で `this` が特定のオブジェクトになり、そのメソッドやプロパティを**名前だけで**呼べるようになります。

```
class Config {
    var mainClass: String = ""
    var fileName: String = "classes.wasm"
}

// 引数の型 Config.() -> Unit が「Config をレシーバにするラムダ」
fun config(block: Config.() -> Unit): Config {
    val c = Config()
    c.block()          // c を this にしてラムダを実行
    return c
}

val cfg = config {
    mainClass = "org.example.NfcWasm"   // this.mainClass = ... の this が省略されている
    fileName = "nfc.wasm"
}
println("${cfg.mainClass} ${cfg.fileName}")   // → org.example.NfcWasm nfc.wasm
```

`teavm { wasmGC { targetFileName = "nfc.wasm" } }` は、まさにこの形です。

- `teavm { ... }` の中では `this` が TeaVM の設定オブジェクト（`TeaVMExtension`）
- `wasmGC { ... }` の中では `this` が Wasm GC の設定オブジェクト（`TeaVMWasmGCConfiguration`）
- だから `targetFileName = ...` と書くだけで、その設定に値が入る

6.3 拡張関数（extension function）

既存のクラスに、あとからメソッドを足したように見せる機能です。

```
fun String.shout(): String = this.uppercase() + "!"
println("wasm".shout())   // → WASM!
```

Gradle の Kotlin DSL は、Java で書かれた Gradle API に大量の拡張関数を足して、Kotlin から書きやすくしています。 `tasks.register<JavaExec>("parseJava") { }` の `register<T>` もそうした拡張関数の 1 つです。

6.4 reified 型パラメータ：`register<JavaExec>`

```
inline fun <reified T> typeName(): String = T::class.simpleName ?: "?"
println(typeName<JavaExec>())   // → JavaExec
```

Java のジェネリクスは実行時に型の情報が消えます（型消去）。Kotlin の `inline` + `reified` を使うと、関数の中で `T` の型情報を使えます。 `tasks.register<JavaExec>("parseJava")` は、内部で `tasks.register("parseJava", JavaExec::class.java)` を呼んでいるのと同じです。

6.5 プロパティへの `=` 代入（Gradle の Property 型）

Gradle の新しい API では、設定値は `Property<String>` のような**遅延評価される箱**で持っています。 本来は `mainClass.set("org.example.NfcWasm")` と書く必要がありますが、Gradle 8.2 以降の Kotlin DSL では **`mainClass = "..."` と `=` で書けます**（代入演算子が `set` に読み替えられる）。

```
mainClass = "org.example.NfcWasm"        // 今の書き方
mainClass.set("org.example.NfcWasm")     // 同じ意味（古い書き方。今でも使える）
```

ネット上の古い記事で `.set(...)` を見かけたら、同じ意味だと思って大丈夫です。

6.6 インデックス演算子：`sourceSets["main"]`

Kotlin では `operator fun get(...)` を定義すると、 `obj[key]` と書けます。 `sourceSets["main"]` は `sourceSets.getByName("main")` と同じ意味です。

6.7 型安全アクセサ（type-safe accessors）

`java { }`、 `teavm { }`、 `teavm.libs.jsoApis`、 `implementation(...)` などは、**プラグインが追加した拡張や構成に対して Gradle が自動生成した Kotlin の関数・プロパティ**です。 `plugins { }` ブロックに書いたプラグインの分だけ生成されるので、IDE で補完が効き、タイプミスはコンパイルエラーになります。 （どんなアクセサが生成されたかは `./gradlew :app:kotlinDslAccessorsReport` で確認できます。）

6.8 スコープ関数（apply / let / also / run / with）

Gradle のスクリプトでもよく見かけるので、2 つだけ覚えておきましょう。

```
val sb = StringBuilder().apply {   // apply：this で設定して、そのオブジェクト自身を返す
    append("A")
    append("B")
}
val len = "kotlin".let { it.length }   // let：it で受け取り、ラムダの結果を返す
println("$sb $len")   // → AB 6
```

6.9 build.gradle.kts を Kotlin の目で読み直す

書かれている形	Kotlin として見ると
`plugins { java }`	`plugins` 関数に、レシーバ付きラムダを渡している。`java` はプラグイン指定用のプロパティ
`id("org.teavm") version "0.16.0"`	`id(...)` の戻り値に対して `version` という**中置関数（infix）**を呼んでいる
`implementation("a:b:1.0")`	`implementation` という生成された関数の呼び出し
`languageVersion = JavaLanguageVersion.of(21)`	`Property` への代入（= `set`）。`of` は Java の static メソッド
`tasks.register<JavaExec>("parseJava") { ... }`	reified な拡張関数 + 末尾ラムダ（中の `this` は `JavaExec`）
`sourceSets["main"].runtimeClasspath`	`get` 演算子 + Java の getter をプロパティとして呼ぶ

---

7. build.gradle.kts を 1 ブロックずつ読む

★★★

7.1 冒頭のコメント

```
/*
 * This file was generated by the Gradle 'init' task.
 * ...
 */
/*
 * https://teavm.org/docs/intro/getting-started.html (2026/10/13時点)
 * 上記サイトを基に作成。ただし、ドキュメントは Groovy の書き方で、このプロジェクトは Kotlin
 */
```

- 1 つ目は `gradle init` コマンドが雛形を作ったときのコメントです。
- 2 つ目は、TeaVM 公式の Getting Started を参考に書き換えたというメモです。**公式ドキュメントは Groovy DSL（`build.gradle`）で書かれている**ため、Kotlin DSL に読み替える必要があります。その対応は 14 章にまとめました。
- コメントの書き方は Java と同じ（`//` と `/* */`）です。

7.2 plugins

```
plugins {
    java
    war
    id("org.teavm") version "0.16.0"
}
```

- `java`：Java のコンパイル・jar 作成などを追加する Gradle 組み込みプラグイン。
- `war`：Web アプリ用の war ファイルを作るプラグイン（`java` の機能も含む）。
- `id("org.teavm") version "0.16.0"`：TeaVM の Gradle プラグイン。外部のプラグインなので ID とバージョンを指定し、Gradle Plugin Portal からダウンロードされる。

詳しくは 8 章。

7.3 repositories

```
repositories {
    mavenCentral()
}
```

ライブラリを **Maven Central**（Java 界最大の公開リポジトリ）からダウンロードする、という指定です。

7.4 dependencies

```
dependencies {
    implementation(teavm.libs.jsoApis)
    implementation("com.github.javaparser:javaparser-core:3.26.4")
    implementation("com.google.code.gson:gson:2.11.0")
}
```

依存	用途	Wasm に入るか
`teavm.libs.jsoApis`	`@JSExport` など、Java と JavaScript をつなぐ API（`org.teavm:teavm-jso-apis:0.16.0`）	使う部分だけ入る
`javaparser-core:3.26.4`	`JavaParser/JavaStructureAnalyzer` で Java ソースを解析する	入らない（コメントのとおり）
`gson:2.11.0`	`JavaApiJsonExporter` で JSON を書き出す	入らない

TeaVM は `mainClass`（`NfcWasm`）から**実際に呼ばれるクラスだけ**をたどって Wasm に入れるので、`NfcWasm` から使われない JavaParser や Gson は Wasm には含まれません。詳しくは 9 章。

7.5 java toolchain

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
```

「Java 21 の JDK でコンパイル・実行してほしい」という指定です。詳しくは 10 章。

7.6 teavm

```
teavm {
    all {
        mainClass = "org.example.NfcWasm"
    }
    wasmGC {
        addedToWebApp = true
        targetFileName = "nfc.wasm"
    }
}
```

- `all { }`：すべてのターゲット（JS・Wasm GC・C）に共通の設定。ここでは入口クラスを `org.example.NfcWasm` にしている。
- `wasmGC { }`：WebAssembly GC 版の設定。
- `addedToWebApp = true`：生成した Wasm を war の中に入れる。
- `targetFileName = "nfc.wasm"`：出力ファイル名。

詳しくは 11 章。

7.7 parseJava タスク

```
tasks.register<JavaExec>("parseJava") {
    group = "application"
    description = "JavaParser で指定フォルダ内の .java ファイルの構造を表示する"
    classpath = sourceSets["main"].runtimeClasspath
    mainClass = "JavaParser.JavaStructureAnalyzer"
    workingDir = rootProject.projectDir
}
```

`JavaParser.JavaStructureAnalyzer` の `main` を実行するための自作タスクです。詳しくは 12 章。

---

8. plugins ブロック：java・war・org.teavm

★★★

8.1 プラグインの種類

種類	書き方	例
コアプラグイン（Gradle に最初から入っている）	`java`、`war`、`` `java-library` ``、`application`	このプロジェクトの `java`・`war`
コミュニティプラグイン（Gradle Plugin Portal から取得）	`id("ID") version "バージョン"`	`id("org.teavm") version "0.16.0"`
Kotlin 公式プラグイン	`kotlin("jvm") version "..."`	Kotlin でアプリを書くとき

コアプラグインは `id("java")` とも書けますが、Kotlin DSL では `java` のように短く書けます。ハイフンを含む名前は `` `java-library` `` のようにバッククォートで囲みます。

8.2 各プラグインが追加するもの

実際に `./gradlew :app:tasks --all` で確認したタスクを、追加したプラグインごとに整理しました。

プラグイン	追加される主なタスク	追加される主な設定
`java`	`compileJava`、`processResources`、`classes`、`jar`、`javadoc`、`test`、`build`、`clean`	`java { }`、`sourceSets`、構成 `implementation` など
`war`	`war`	`src/main/webapp` を war に入れる。構成 `providedCompile` / `providedRuntime`
`org.teavm`	`generateWasmGC`、`buildWasmGC`、`wasmGCDevServer`、`generateJavaScript`、`javaScriptDevServer`、`stopDevServer`、`generateC`、`disasmWasmGC`、`copyWasmGCRuntime` など	`teavm { }`、ソースセット `teavm`（`compileTeavmJava` タスク）

8.3 war プラグインと「webapp」

war（Web Application Archive）は、本来は Java の Web サーバー（Tomcat など）に置くための zip 形式のファイルです。 このプロジェクトでは**サーバーに置くためではなく、「静的ファイル一式をまとめる入れ物」として使っています**。実際にできた `app/build/libs/app.war` の中身（クラスファイルを除く）は次のとおりです。

```
index.html
nfc.js
nfc.generated.js
wasm-gc/nfc.wasm
wasm-gc/nfc.wasm-runtime.js
WEB-INF/lib/teavm-jso-apis-0.16.0.jar
WEB-INF/lib/javaparser-core-3.26.4.jar
WEB-INF/lib/gson-2.11.0.jar
...
```

- `src/main/webapp/` の中身（`index.html`、`nfc.js` など）がそのままルートに入る。
- TeaVM が作った `nfc.wasm` とランタイム JS が `wasm-gc/` に入る（`addedToWebApp = true` のおかげ）。
- `WEB-INF/lib/` には依存 jar が入るが、ブラウザでは使われない。

war は zip なので、展開して `index.html` をローカルサーバーで開けば動作確認ができます（このプロジェクトの `out/` はそのためのフォルダ）。

---

9. repositories と dependencies：ライブラリの取り込み

★★★

9.1 依存の書き方（座標）

```
implementation("com.google.code.gson:gson:2.11.0")
//             └── group ──────────┘  └artifact┘  └ver┘
```

Maven と同じ **`group:artifact:version`** の 3 つ組でライブラリを指定します。 ライブラリの座標は Maven Central の検索 で調べられます。

9.2 構成（configuration）の種類

`implementation` の部分を**構成**と呼び、「その依存をいつ使うか」を表します。

構成	コンパイル時	実行時	使いどころ
`implementation`	○	○	普通はこれ
`compileOnly`	○	×	コンパイルにだけ必要（アノテーションなど）
`runtimeOnly`	×	○	実行時にだけ必要（JDBC ドライバなど）
`testImplementation`	テストのみ	テストのみ	JUnit など
`api`	○	○	`java-library` プラグインで使える。利用側にも依存を公開する
`providedCompile`	○	war に入れない	`war` プラグイン。サーバーが提供するライブラリ

9.3 推移的依存

ライブラリが別のライブラリに依存していると、それも自動で取り込まれます（**推移的依存**）。 このプロジェクトの実行時クラスパスを `./gradlew :app:dependencies --configuration runtimeClasspath` で表示すると、次のようになりました。

```
runtimeClasspath - Runtime classpath of source set 'main'.
+--- org.teavm:teavm-jso-apis:0.16.0
|    +--- org.teavm:teavm-jso:0.16.0
|    ¥--- org.teavm:teavm-interop:0.16.0
+--- com.github.javaparser:javaparser-core:3.26.4
¥--- com.google.code.gson:gson:2.11.0
     ¥--- com.google.errorprone:error_prone_annotations:2.27.0
```

3 つ書いただけで、6 つの jar が取り込まれていることがわかります。

9.4 `teavm.libs.jsoApis` とは

文字列ではなく、TeaVM プラグインが用意した**依存の定数**です。TeaVM プラグインのバージョン（0.16.0）と同じバージョンのライブラリを自動で指してくれるので、バージョンのずれが起きません。

プラグイン本体を調べると、`teavm.libs` には次の 5 つが用意されています。

アクセサ	中身
`teavm.libs.jso`	Java ↔ JS 連携の基本（`@JSExport`、`@JSBody` など）
`teavm.libs.jsoApis`	ブラウザ API（DOM など）の Java 定義。`jso` も含む
`teavm.libs.interop`	低レベルの相互運用
`teavm.libs.metaprogramming`	TeaVM のメタプログラミング API
`teavm.libs.spi`	TeaVM の拡張用 SPI

9.5 ダウンロードしたライブラリの保存場所

ライブラリは `~/.gradle/caches/` に保存され、すべてのプロジェクトで共有されます。2 回目以降はネットに接続しなくても使えます（`--offline` オプション）。

---

10. java toolchain と JDK の指定

★★

10.1 2 種類の「JDK の指定」がある

このプロジェクトには JDK の指定が 2 か所あり、役割が違います。

指定場所	何の JDK か	値
`gradle.properties` の `org.gradle.java.home`	**Gradle 自身（デーモン）を動かす** JDK	`/usr/lib/jvm/java-21-openjdk-amd64`
`build.gradle.kts` の `java { toolchain { } }`	**コンパイルや JavaExec に使う** JDK	Java 21

どちらも 21 なので現状は同じ JDK が使われますが、別々に変えることもできます。

10.2 toolchain の利点

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
```

- PC にどの JDK がデフォルトで入っていても、**必ず Java 21 でコンパイル**される。
- 該当する JDK が見つからなければ、`settings.gradle.kts` の foojay プラグインが**自動でダウンロード**する。
- 見つかった JDK は `./gradlew :app:javaToolchains` で確認できる。

10.3 Java 21 を使う理由

TeaVM 0.16.0 は Java のバイトコード（.class）を読んで変換します。JDK 21 は長期サポート版（LTS）で、`record` やパターンマッチなどの新しい文法も使えます。 ただしプロジェクトの決まりとして、Java コードでは `var` を使わず明示的な型で書く方針です（CLAUDE.md）。

---

11. teavm ブロック：Wasm を作る設定

★★★

11.1 TeaVM とは

TeaVM は、Java のバイトコード（.class）を **JavaScript・WebAssembly・C** に変換するコンパイラです。`mainClass` から呼ばれるクラス・メソッドだけをたどって変換するので（到達可能性解析）、約 400 ファイルある NFC ライブラリでも、必要な部分だけが Wasm に入ります。

11.2 ターゲットの種類

`teavm { }` の中には、ターゲットごとのブロックがあります（プラグイン本体の `TeaVMExtension` で確認）。

ブロック	出力	実行タスク
`all { }`	全ターゲット共通の設定	—
`js { }`	JavaScript	`generateJavaScript`
`wasmGC { }`	WebAssembly（GC 対応版）	`generateWasmGC` / `buildWasmGC`
`c { }`	C 言語のソース	`generateC`

WasmGC は、WebAssembly にガベージコレクション（GC：使わなくなったメモリの自動回収）の仕組みを加えた新しい規格です。Java のオブジェクトをブラウザの GC に任せられるので、Wasm が小さく速くなります。現行の主要ブラウザ（Chrome・Firefox・Safari）の新しいバージョンで動きます。

11.3 このプロジェクトの設定

設定	値	意味
`all.mainClass`	`"org.example.NfcWasm"`	変換の入口。`@JSExport` を付けた static メソッドが JS から呼べる
`wasmGC.addedToWebApp`	`true`	生成物を war の `wasm-gc/` に入れる
`wasmGC.targetFileName`	`"nfc.wasm"`	出力ファイル名。ランタイムは `nfc.wasm-runtime.js` になる

生成物は `app/build/generated/teavm/wasm-gc/` に出力され、そこから war に入ります。

11.4 使える主な設定項目

TeaVM Gradle プラグイン 0.16.0 の jar を `javap` で調べ、設定できる項目を一覧にしました（★は覚えておくと便利なもの）。

全ターゲット共通（`all { }` やそれぞれのブロックで使える）

項目	型	説明
★ `mainClass`	String	入口クラス
★ `optimization`	`OptimizationLevel`（`NONE` / `BALANCED` / `AGGRESSIVE`）	最適化の強さ
`debugInformation`	Boolean	デバッグ情報を出す
`fastGlobalAnalysis`	Boolean	解析を速くする（最適化は弱くなる）
`preservedClasses`	List	到達しなくても残すクラス
`properties`	Map	TeaVM に渡す追加プロパティ
`outOfProcess` / `processMemory`	Boolean / Int	別プロセスで変換する／そのメモリ量
`outputDir`	Directory	出力先フォルダ

`wasmGC { }` 用

項目	型	説明
★ `targetFileName`	String	出力ファイル名
★ `addedToWebApp`	Boolean	war に入れるか
`relativePathInOutputDir`	String	出力フォルダ内の相対パス（今の出力先 `wasm-gc/` を変えたいときに使うと考えられる。未検証）
★ `obfuscated`	Boolean	名前を短くしてサイズを減らす
`strict`	Boolean	配列の範囲チェックなどを厳密に行う
`copyRuntime`	Boolean	`*-runtime.js` をコピーするか
★ `sourceMap`	Boolean	ブラウザで Java ソースを見ながらデバッグするためのソースマップ
`debugInfoLevel` / `debugInfoLocation`	`FULL`・`DEOBFUSCATION` / `EMBEDDED`・`EXTERNAL`	デバッグ情報の量と置き場所
`disassembly`	Boolean	Wasm を読める形（テキスト）でも出力
`modularRuntime`	Boolean	ランタイムを ES モジュール形式にする
`importedWasmMemory` / `sharedBuffer`	Boolean	メモリの扱いに関する上級者向け設定
`devServer { }`	ブロック	`wasmGCDevServer` の設定（ポート番号など）

たとえば、デバッグしやすくしたいときと、配布用に小さくしたいときは次のように書き分けられます。

```
teavm {
    all {
        mainClass = "org.example.NfcWasm"
        optimization = org.teavm.gradle.api.OptimizationLevel.AGGRESSIVE   // 配布用
    }
    wasmGC {
        addedToWebApp = true
        targetFileName = "nfc.wasm"
        obfuscated = true        // 配布用：サイズを小さく
        // sourceMap = true      // デバッグ用：Java ソースでデバッグ
    }
}
```

（このコード例は設定項目の名前と型をプラグインの API で確認したもので、実際に変換したときのサイズや挙動の違いは試していません。）

---

12. カスタムタスク parseJava

★★

12.1 何をするタスクか

```
// Java ファイルの構造を表示する: ./gradlew :app:parseJava --args="<フォルダ>"
// JSON で書き出す:           ./gradlew :app:parseJava --args="<フォルダ> --json <出力ファイル>"
tasks.register<JavaExec>("parseJava") {
    group = "application"
    description = "JavaParser で指定フォルダ内の .java ファイルの構造を表示する"
    classpath = sourceSets["main"].runtimeClasspath
    mainClass = "JavaParser.JavaStructureAnalyzer"
    workingDir = rootProject.projectDir
}
```

`JavaExec` は「Java のクラスの main メソッドを実行する」ための、Gradle 組み込みのタスクの型です。

行	意味
`tasks.register<JavaExec>("parseJava")`	`JavaExec` 型の `parseJava` タスクを登録する
`group = "application"`	`./gradlew tasks` で表示されるときのグループ名
`description = ...`	タスクの説明（`tasks` の一覧に出る）
`classpath = sourceSets["main"].runtimeClasspath`	`main` のコンパイル結果 + 実行時の依存（JavaParser・Gson）をクラスパスにする
`mainClass = "JavaParser.JavaStructureAnalyzer"`	実行するクラス（パッケージ `JavaParser` の `JavaStructureAnalyzer`）
`workingDir = rootProject.projectDir`	作業フォルダを `make_wasm/TeaVM/` にする。だから `--args` に `app/src/...` という相対パスを渡せる

12.2 使い方

```
# 構造を表示する
./gradlew :app:parseJava --args="app/src/main/java/org/mklab/nfc"

# JSON に書き出す（JavaGenerator.py の入力になる）
./gradlew :app:parseJava --args="app/src/main/java/org/mklab/nfc --json build/nfc_api.json"
```

`--args` は `JavaExec` タスクが持つコマンドラインオプションで、main メソッドの `String[] args` に渡されます。

12.3 register と create の違い

書き方	タスクが作られるタイミング
`tasks.register("x") { }`	**必要になったときだけ**作られる（遅延）。今の推奨
`tasks.create("x") { }`	すぐに作られる。古い書き方

`register` を使うと、使わないタスクの設定処理が省かれてビルドが速くなります。

12.4 `register` でできる他のタスクの型

型	用途
`JavaExec`	Java の main を実行
`Exec`	外部コマンド（python3 など）を実行
`Copy`	ファイルをコピー
`Delete`	ファイルを削除
`Zip` / `Jar`	アーカイブを作る
型なし（`DefaultTask`）	`doLast { }` に好きな処理を書く

---

13. ./gradlew :app:build で実際に起きること

★★★

`--dry-run`（`-m`）を付けると、**実行せずに、実行されるタスクの順番だけ**を表示できます。

```
./gradlew :app:build --dry-run
```

実際の出力（抜粋）：

```
:app:compileJava SKIPPED
:app:processResources SKIPPED
:app:classes SKIPPED
:app:compileTeavmJava SKIPPED
:app:processTeavmResources SKIPPED
:app:teavmClasses SKIPPED
:app:generateWasmGC SKIPPED
:app:disasmWasmGC SKIPPED
:app:emscriptenStubWasmGC SKIPPED
:app:emscriptenWasmGC SKIPPED
:app:buildWasmGC SKIPPED
:app:war SKIPPED
:app:assemble SKIPPED
:app:compileTestJava SKIPPED
:app:processTestResources SKIPPED
:app:testClasses SKIPPED
:app:test SKIPPED
:app:check SKIPPED
:app:build SKIPPED
```

（`SKIPPED` は dry-run なので実行していない、という意味です。）

各タスクの意味

タスク	追加したプラグイン	すること
`compileJava`	java	`src/main/java` の .java を .class にコンパイル
`processResources`	java	`src/main/resources` をコピー
`classes`	java	上 2 つをまとめる目印のタスク
`compileTeavmJava` 〜 `teavmClasses`	teavm	TeaVM プラグインが追加するソースセット `teavm` の処理。このプロジェクトでは使っていない
`generateWasmGC`	teavm	**.class → nfc.wasm の変換本体**
`disasmWasmGC` / `emscripten*`	teavm	逆アセンブル・Emscripten 連携。設定していないので実質何もしない
`buildWasmGC`	teavm	Wasm 関連の生成をまとめる
`war`	war	webapp + Wasm + クラスを `app.war` にまとめる
`assemble`	java	成果物を作るタスクのまとめ
`compileTestJava` 〜 `test`	java	テストのコンパイルと実行（テストはまだないので空）
`check`	java	検証系タスクのまとめ
`build`	java	`assemble` + `check`

目的別に、どのタスクを呼べばよいか

やりたいこと	コマンド
全部（war まで）	`./gradlew :app:build`
テストを飛ばして war まで	`./gradlew :app:assemble` または `./gradlew :app:war`
Wasm だけ作りたい	`./gradlew :app:generateWasmGC`
Java のコンパイルが通るかだけ確認	`./gradlew :app:compileJava`

---

14. Groovy DSL との対応表

★★

TeaVM の公式ドキュメントや古い記事は、**Groovy DSL**（`build.gradle`）で書かれていることが多いです。Kotlin DSL（`build.gradle.kts`）に直すときの対応表です。

内容	Groovy（build.gradle）	Kotlin（build.gradle.kts）
文字列	`'abc'` も `"abc"` も可	**`"abc"` のみ**（`'a'` は 1 文字の Char）
プラグイン	`id 'java'`	`java` または `id("java")`
外部プラグイン	`id 'org.teavm' version '0.16.0'`	`id("org.teavm") version "0.16.0"`
依存	`implementation 'g:a:v'`	`implementation("g:a:v")`（**括弧が必須**）
TeaVM の依存	`implementation teavm.libs.jsoApis`	`implementation(teavm.libs.jsoApis)`
プロパティ代入	`mainClass = "X"`	`mainClass = "X"`（同じ）
タスク登録	`tasks.register('parseJava', JavaExec) { }`	`tasks.register<JavaExec>("parseJava") { }`
ソースセット	`sourceSets.main.runtimeClasspath`	`sourceSets["main"].runtimeClasspath`
既存タスクの設定	`tasks.named('war') { }` または `war { }`	`tasks.named("war") { }` または `tasks.war { }`
変数	`def x = 1`	`val x = 1`
関数呼び出しの括弧	省略できる	省略できない（末尾ラムダのみ外に出せる）

TeaVM 公式（Groovy）の書き方と、このプロジェクト（Kotlin）の比較：

```
// Groovy 版 (build.gradle)
plugins {
    id 'java'
    id 'war'
    id 'org.teavm' version '0.16.0'
}
dependencies {
    implementation teavm.libs.jsoApis
}
teavm {
    all {
        mainClass = "org.example.NfcWasm"
    }
    wasmGC {
        addedToWebApp = true
    }
}
```

```
// Kotlin 版 (build.gradle.kts)
plugins {
    java
    war
    id("org.teavm") version "0.16.0"
}
dependencies {
    implementation(teavm.libs.jsoApis)
}
teavm {
    all {
        mainClass = "org.example.NfcWasm"
    }
    wasmGC {
        addedToWebApp = true
    }
}
```

Kotlin DSL の利点

利点	説明
IDE の補完が効く	`teavm {` `wasmGC {` まで打つと、設定できる項目が候補に出る
間違いがすぐわかる	存在しない設定名や型の違いはコンパイルエラーになる
定義にジャンプできる	Ctrl+クリックで、プラグインのソース（API）を見られる
新規プロジェクトの標準	Gradle 8.2 以降、`gradle init` の既定は Kotlin DSL

欠点は、初回のスクリプトのコンパイルが Groovy より少し遅いことです（構成キャッシュである程度解消されます）。

---

15. よく使う Gradle コマンド集

★★★

コマンドは `make_wasm/TeaVM/` で実行します（WSL の Ubuntu 上）。

基本

コマンド	意味
`./gradlew :app:build`	ビルド（war まで）
`./gradlew :app:clean`	`app/build/` を削除
`./gradlew :app:clean :app:build`	きれいにしてからビルド
`./gradlew :app:tasks`	主なタスク一覧（`--all` で全部）
`./gradlew help --task :app:parseJava`	タスクの詳しい説明
`./gradlew --version`	Gradle・Kotlin・JVM のバージョン

調べる

コマンド	意味
`./gradlew :app:dependencies`	依存ツリー
`./gradlew :app:dependencies --configuration runtimeClasspath`	実行時クラスパスだけ
`./gradlew :app:dependencyInsight --dependency gson`	ある依存がなぜ入っているか
`./gradlew :app:javaToolchains`	見つかった JDK
`./gradlew :app:build --dry-run`	実行されるタスクの順番
`./gradlew :app:properties`	プロジェクトのプロパティ一覧

よく使うオプション

オプション	意味
`--info` / `--debug`	詳しいログ
`--stacktrace`	エラー時にスタックトレース
`--scan`	Web 上に詳しいレポート（外部サービスに送信されるので注意）
`--offline`	ネットに接続しない
`--rerun-tasks`	UP-TO-DATE を無視して全部やり直す
`--continuous`（`-t`）	ファイルが変わるたびに自動で再ビルド
`-q`	出力を減らす
`--no-configuration-cache`	構成キャッシュを使わない（不具合の切り分けに）
`-P名前=値`	プロジェクトプロパティを渡す

Gradle デーモン

コマンド	意味
`./gradlew --stop`	常駐している Gradle デーモンを止める
`./gradlew --status`	デーモンの状態

Gradle は 2 回目以降を速くするため、裏で**デーモン**（常駐プロセス）を動かし続けます。設定がおかしくなったと感じたら `--stop` で止めると直ることがあります。

---

16. このビルドを拡張する例

★★

ここでは「こう書けばこんなこともできる」という例を示します。**このプロジェクトのファイルは変更していません。** `Exec` ・ `Delete` タスクの書き方は別の一時プロジェクトで実行して動くことを確かめました。

16.1 コード生成を Gradle のタスクにする

今は `python3 app/src/main/Python/JavaGenerator.py` を手で実行していますが、Gradle のタスクにできます。

```
// parseJava で JSON を作ってから JavaGenerator.py を実行する
val generateBridges by tasks.registering(Exec::class) {
    group = "codegen"
    description = "nfc_api.json からブリッジと JS ラッパーを生成する"
    workingDir = rootProject.projectDir
    commandLine("python3", "app/src/main/Python/JavaGenerator.py")
}
```

- `val 名前 by tasks.registering(型::class) { }` は、タスクを登録しつつ同じ名前の変数にも入れる書き方です（**委譲プロパティ**という Kotlin の機能）。
- `commandLine(...)` に外部コマンドを渡します。

16.2 タスクの依存関係を追加する

```
tasks.named("assemble") {
    dependsOn(generateBridges)   // assemble の前に必ず generateBridges を実行する
}
```

書き方	意味
`dependsOn(x)`	自分の前に x を実行する（x は必ず実行される）
`mustRunAfter(x)`	両方実行されるなら x の後にする（x を実行させはしない）
`finalizedBy(x)`	自分の後に必ず x を実行する

16.3 古い Wasm を消すタスク

```
tasks.register<Delete>("cleanWasm") {
    group = "build"
    delete(layout.buildDirectory.dir("generated/teavm/wasm-gc"))
}
```

`layout.buildDirectory` は `app/build/` を指します。パスを直接書かずにこう書くと、ビルドフォルダの場所を変えても壊れません（18 章の「古い Wasm が war に残る」問題の対策になります）。

16.4 doLast：実行フェーズで動く処理

```
tasks.register("hello") {
    group = "custom"
    description = "あいさつを表示する"
    doLast {
        println("Hello from Gradle task")   // ./gradlew hello のときだけ表示される
    }
}
```

`doLast { }` の外に `println` を書くと、**どのタスクを実行しても設定フェーズで表示されてしまいます**。作業は必ず `doLast { }`（または `doFirst { }`）の中に書きます。

16.5 コマンドラインから値を渡す

```
val greetName = providers.gradleProperty("who").orElse("World")
tasks.register("greet") {
    val w = greetName
    doLast { println("Hello, ${w.get()}") }
}
```

```
./gradlew greet -Pwho=NFC    # → Hello, NFC
```

構成キャッシュが有効なときは、`doLast` の中から `project` を直接触るとエラーになることがあります。上のように**外で値（Provider）を取り出してから** `doLast` で使うのが安全です。

16.6 バージョンカタログを使う

`gradle/libs.versions.toml` に書くと、

```
[versions]
javaparser = "3.26.4"
gson = "2.11.0"

[libraries]
javaparser-core = { module = "com.github.javaparser:javaparser-core", version.ref = "javaparser" }
gson = { module = "com.google.code.gson:gson", version.ref = "gson" }
```

build.gradle.kts では型安全に参照できます（`-` は `.` に変わります）。

```
dependencies {
    implementation(libs.javaparser.core)
    implementation(libs.gson)
}
```

16.7 解析ツールと Wasm 用コードを分ける（発展）

今は JavaParser・Gson と Wasm 用のコードが同じ `main` ソースセットにあるため、war の `WEB-INF/lib` に不要な jar が入ります（Wasm には入りません）。 気になる場合は、解析ツールを別のソースセットや別のサブプロジェクト（`settings.gradle.kts` に `include("tools")` を追加）に分ける方法があります。ただし構成が複雑になるので、困っていなければ今のままで問題ありません。

---

17. Kotlin で何ができるか

★★

Kotlin はビルドスクリプト専用の言語ではなく、さまざまな場所で使われている汎用のプログラミング言語です。

17.1 Kotlin の主な用途

分野	内容	代表的なもの
Android アプリ	Google が 2019 年に「Kotlin ファースト」を表明し、Android 開発の第一言語になった	Jetpack Compose（UI）
サーバーサイド	JVM で動くので Java のライブラリ資産をそのまま使える	Spring Boot（Kotlin 対応）、Ktor（JetBrains 製）
マルチプラットフォーム	1 つのコードを Android・iOS・デスクトップ・Web で共有	Kotlin Multiplatform（KMP）、Compose Multiplatform
Web（JavaScript）	Kotlin を JavaScript に変換	Kotlin/JS
Web（WebAssembly）	Kotlin を WasmGC に変換。このプロジェクトと同じ WasmGC を使う	Kotlin/Wasm
ネイティブ	JVM なしで動く実行ファイルを作る（iOS など）	Kotlin/Native
ビルドスクリプト	Gradle の設定	`build.gradle.kts`
スクリプト	シェルスクリプトの代わり	`*.main.kts`
データ分析	ノートブックでの分析	Kotlin Notebook、Kotlin DataFrame
デスクトップアプリ	GUI アプリ	Compose for Desktop

17.2 Java との相互運用

Kotlin の大きな強みは、**Java と同じプロジェクトで混ぜて使える**ことです。

- Kotlin から Java のクラス（たとえば NFC の `DoubleMatrix`）をそのまま呼べる。
- Java から Kotlin のクラスも呼べる。
- 同じ .class ファイル（JVM バイトコード）になるので、TeaVM のような .class を読むツールとも理屈の上では組み合わせられる（ただし Kotlin の標準ライブラリを TeaVM が扱えるかは別途確認が必要）。

17.3 このプロジェクトとの関係：Kotlin/Wasm という別ルート

このプロジェクトは「**Java → TeaVM → WasmGC**」というルートです。似たことを Kotlin で行う「**Kotlin → Kotlin/Wasm → WasmGC**」というルートもあります。

比べる点	TeaVM（このプロジェクト）	Kotlin/Wasm
入力	Java のバイトコード（.class）	Kotlin のソースコード
既存の Java ライブラリ（NFC）	**そのまま使える**	Kotlin で書き直すか、Kotlin Multiplatform 向けに移植が必要
JS との連携	`@JSExport` / `@JSBody`	`@JsExport` / `external` 宣言
ビルド	Gradle + TeaVM プラグイン	Gradle + Kotlin Multiplatform プラグイン
成熟度	長く開発されている	公式ドキュメント上は発展途中の段階（状態は変わりやすいので公式サイトで確認）

NFC のような既存の Java ライブラリを**変更せずに**ブラウザで動かしたい、という目的には TeaVM が合っています。

17.4 Kotlin を試すなら

- Kotlin Playground：ブラウザだけで Kotlin を書いて実行できる。5 章のコード例を貼り付けて試せる。
- IntelliJ IDEA（Community 版は無料）：Kotlin を作った JetBrains 社の IDE。Kotlin と Gradle のサポートが最も充実している。
- VS Code：Kotlin の拡張機能はあるが、IntelliJ ほど補完は強くない。build.gradle.kts を書くなら IntelliJ が快適。

---

18. よくある失敗・ハマりどころ

★★★

18.1 古い Wasm が war に残る

実際に今の `app/build/generated/teavm/wasm-gc/` と war を確認すると、**以前の設定で作った `example.wasm` や `eig_example.wasm` が残ったまま war に入っています。** `targetFileName` を変えても、古いファイルは自動では消えません。

- 対策：`./gradlew :app:clean :app:build` で一度きれいにする。
- 対策：16.3 の `cleanWasm` のようなタスクを作る。

18.2 Groovy の書き方をそのまま貼る

Groovy のまま貼ると	エラーの原因	Kotlin での正解
`implementation 'g:a:v'`	`'...'` は文字列ではない／括弧がない	`implementation("g:a:v")`
`id 'org.teavm' version '0.16.0'`	同上	`id("org.teavm") version "0.16.0"`
`sourceSets.main.runtimeClasspath`	`main` というプロパティはない	`sourceSets["main"].runtimeClasspath`
`tasks.register('x', JavaExec)`	クラスの書き方が違う	`tasks.register<JavaExec>("x")`

18.3 doLast を忘れる

```
tasks.register("bad") {
    println("毎回表示される！")   // 設定フェーズで実行される
}
```

どんなタスクを実行しても表示されてしまいます。作業は `doLast { }` の中に書きます（16.4）。

18.4 構成キャッシュ関連のエラー

`gradle.properties` で構成キャッシュを有効にしているため、`doLast` の中で `project.xxx` を使うと「configuration cache problems」というエラーが出ることがあります。

- 対策：値は設定フェーズで変数や `Provider` に取り出しておく（16.5）。
- 一時的に `--no-configuration-cache` を付けて、原因が構成キャッシュかどうか切り分ける。

18.5 IDE で build.gradle.kts が赤くなる

`teavm` や `implementation` が赤字（未解決）になるのは、IDE がまだ Gradle の同期（Sync）をしていないためであることが多いです。

- IntelliJ：Gradle ツールウィンドウの「Reload」。
- VS Code：Gradle 拡張や Java 拡張の「プロジェクトを再読み込み」。
- WSL のフォルダを Windows 側の IDE で開くと、JDK のパス（`/usr/lib/jvm/...` は Linux のパス）が解決できず赤くなることがある。ビルド自体は WSL のターミナルで `./gradlew` を使えば問題ない。

18.6 JDK のバージョンが合わない

- `org.gradle.java.home` に書いたパスが存在しないと、Gradle が起動しない。別の PC にコピーしたときに起きやすい。
- 「Unsupported class file major version」：Gradle やプラグインが対応していない新しすぎる JDK を使っている。

18.7 gradlew が実行できない（WSL）

- `Permission denied`：実行権限がない → `chmod +x gradlew`
- `/usr/bin/env: 'sh¥r': No such file or directory`：Windows 側で改行コードが CRLF に変わった → Git の `core.autocrlf` を見直す、または LF に戻す。

18.8 `:app:` を付け忘れる

ルートで `./gradlew parseJava` と打っても、Gradle は全プロジェクトから同名のタスクを探すので多くの場合は動きます。ただし、どのプロジェクトのタスクかを明確にするため `:app:parseJava` と書くのが確実です。

18.9 `--args` の引用符

```
./gradlew :app:parseJava --args="app/src/main/java/org/mklab/nfc --json build/nfc_api.json"
```

複数の引数は、全体を 1 つの `"..."` で囲みます。囲まないと、`--json` が Gradle 自身のオプションとして解釈されてエラーになります。

18.10 mainClass が 2 か所にある

場所	何の mainClass か
`teavm { all { mainClass = ... } }`	TeaVM が Wasm に変換するときの入口（`org.example.NfcWasm`）
`tasks.register<JavaExec>("parseJava") { mainClass = ... }`	`parseJava` で JVM 上で実行するクラス（`JavaParser.JavaStructureAnalyzer`）

名前は同じですが、別の設定です。自動生成ブリッジ（`org.example.generated.NfcWasm`）に切り替えたいときに変えるのは、`teavm` の方です。

---

19. 学習の進め方と参考資料

★★

19.1 おすすめの順番

段階	やること	目安
1	このレポートの 1・7 章で、ファイルの全体像をつかむ	30 分
2	Kotlin Playground で 5・6 章のコードを動かす。特に「レシーバ付きラムダ」	1〜2 時間
3	`./gradlew :app:tasks --all`、`dependencies`、`--dry-run` を実際に打つ	30 分
4	16 章の `hello` タスクを自分で足して動かしてみる（試したら戻す）	30 分
5	TeaVM の公式ドキュメント（Groovy）を、14 章の表を見ながら読む	1 時間
6	Kotlin Koans で Kotlin の文法を一通り練習する	数時間

19.2 参考資料（すべて 2026 年 11 月にアクセスできることを確認）

資料	URL	内容
TeaVM Getting Started	https://teavm.org/docs/intro/getting-started.html	build.gradle.kts の元になった公式手順（Groovy）
TeaVM Gradle プラグイン	https://teavm.org/docs/tooling/gradle.html	`teavm { }` の設定の説明
TeaVM プラグイン（Plugin Portal）	https://plugins.gradle.org/plugin/org.teavm	バージョン一覧
Gradle ユーザーガイド（8.14.4）	https://docs.gradle.org/8.14.4/userguide/userguide.html	このプロジェクトと同じバージョンの公式ドキュメント
Gradle Kotlin DSL 入門	https://docs.gradle.org/current/userguide/kotlin_dsl.html	Kotlin DSL の書き方、Groovy との違い
War プラグイン	https://docs.gradle.org/current/userguide/war_plugin.html	war の構成
Toolchains	https://docs.gradle.org/current/userguide/toolchains.html	JDK の指定方法
Configuration Cache	https://docs.gradle.org/current/userguide/configuration_cache.html	構成キャッシュの制約
Kotlin 公式ドキュメント	https://kotlinlang.org/docs/home.html	言語の公式リファレンス
Kotlin 基本文法	https://kotlinlang.org/docs/basic-syntax.html	5 章の内容の公式版
Kotlin Koans	https://kotlinlang.org/docs/koans.html	穴埋め形式の練習問題
Kotlin Playground	https://play.kotlinlang.org/	ブラウザで Kotlin を実行
Kotlin/Wasm	https://kotlinlang.org/docs/wasm-overview.html	17.3 の Kotlin/Wasm の説明
Maven Central	https://central.sonatype.com/	ライブラリの座標を調べる
Gradle の解説記事（Zenn）	https://zenn.dev/suwash/articles/gradle_20250905	build.gradle.kts のコメントで参照されている記事

20. チェックリスト

読み終えたら、次のことができるか確認してみましょう。

Kotlin

- ☐ `val` と `var` の違いを説明できる
- ☐ `String` と `String?` の違い、`?.` と `?:` の意味を説明できる
- ☐ `"${name.length}"` のような文字列テンプレートが読める
- ☐ 末尾ラムダで `f({ ... })` が `f { ... }` になることを説明できる
- ☐ レシーバ付きラムダ（`T.() -> Unit`）の中で `this` が何になるか説明できる

Gradle

- ☐ 初期化・設定・実行の 3 フェーズを説明できる
- ☐ `settings.gradle.kts`・`build.gradle.kts`・`gradle.properties` の役割を区別できる
- ☐ `implementation` と `compileOnly` の違いを説明できる
- ☐ `./gradlew :app:dependencies` で依存ツリーを表示できる
- ☐ `./gradlew :app:build --dry-run` で実行順を確認できる
- ☐ 作業を `doLast { }` に書く理由を説明できる

このプロジェクト

- ☐ `teavm.all.mainClass` を変えると何が変わるか説明できる
- ☐ `nfc.wasm` がどこに出力され、war のどこに入るか言える
- ☐ `parseJava` に `--args` で引数を渡して実行できる
- ☐ 古い Wasm が war に残る理由と、その消し方を説明できる
- ☐ TeaVM の公式ドキュメント（Groovy）を Kotlin DSL に読み替えられる

---

21. まとめ

- `build.gradle.kts` は **Kotlin で書かれた Gradle の設定**で、このプロジェクトでは「Java をコンパイル → TeaVM で `nfc.wasm` に変換 → webapp と一緒に war にまとめる」流れを 58 行で宣言している。
- **Gradle** は依存ライブラリの管理とタスクの自動実行を担うビルドツール。設定フェーズで build.gradle.kts を実行してタスクグラフを作り、実行フェーズで必要なタスクを依存順に実行する。
- **Kotlin DSL** が設定ファイルらしく見えるのは、**末尾ラムダ・レシーバ付きラムダ・拡張関数・Property への `=` 代入**のおかげ。これを知っていれば、`teavm { wasmGC { targetFileName = "nfc.wasm" } }` が普通の Kotlin の関数呼び出しだとわかる。
- `teavm { }` では、入口クラス（`mainClass`）と出力（`targetFileName`・`addedToWebApp`）を指定している。他にも `optimization`・`obfuscated`・`sourceMap` などの設定がある。

- `parseJava` は `JavaExec` 型の自作タスクで、同じやり方で `Exec`（Python 実行）や `Delete`（掃除）のタスクも足せる。
- **Kotlin** は Android・サーバー・マルチプラットフォーム・Web（JS / Wasm）・スクリプトまで使える汎用言語で、Java と混ぜて使える。既存の Java ライブラリを変えずにブラウザで動かすという目的には、このプロジェクトの TeaVM ルートが合っている。
- ハマりやすいのは、**古い Wasm が war に残ること、Groovy の書き方の貼り付け、`doLast` の書き忘れ、構成キャッシュ**。困ったら `clean`・`--dry-run`・`--no-configuration-cache`・`--stop` で切り分ける。